AI QUANT RESEARCH LAB

End-to-End Technical, Product, Trading, AI, Risk, Data, Infrastructure & Execution Documentation

Project concept: Build an AI-native quantitative research platform that systematically generates trading hypotheses, tests them against clean historical data, validates them out-of-sample, paper trades approved strategies, and—only after rigorous validation—supports controlled live execution.

This document is a complete blueprint for the project discussed in this conversation. It covers the business model, architecture, technology stack, team, capital, data, broker/exchange access, AI architecture, quantitative research workflow, backtesting, risk, security, deployment, testing, operations, roadmap, and realistic first product.

Table of Contents

1. Executive Summary

The AI Quant Research Lab is an AI-native quantitative research and trading platform. Its first purpose is not to promise market-beating returns; its purpose is to create a disciplined scientific process for discovering, testing, rejecting, validating, and monitoring systematic trading ideas.

The core product is a research loop: market data → hypothesis → strategy code → backtest → statistical validation → out-of-sample test → risk analysis → paper trading → human approval → controlled execution.

The LLM is not the final authority for trades. AI accelerates research, coding, literature review, experiment design, and analysis. Deterministic quantitative systems, independent risk controls, and human approval remain responsible for capital deployment.

- Python for research/AI; C++ for latency-sensitive market-data/execution components.
- PostgreSQL + Parquet/object storage, Redis, FastAPI, Docker, cloud compute, observability and CI/CD.
- AI agents for research, coding, experimentation, statistical review, risk review and monitoring.
- Reproducible experiments, realistic backtesting, walk-forward validation and transaction-cost modeling.

2. Product Vision and Scope

The vision is to create an AI-native quantitative research organization in software. Instead of researchers manually searching thousands of ideas, the platform continuously turns research questions into testable hypotheses and experiments.

Example: 'Investigate whether mean reversion among liquid equities remains robust after transaction costs and across multiple market regimes.' The system decomposes the request into data requirements, hypotheses, experiments, statistical tests and a final report.

Every experiment must be reproducible and traceable to data versions, code, parameters, market universe, timestamps, costs and model versions.

- Research management and experiment tracking.
- Historical and real-time market-data pipelines.
- Backtesting and simulation.
- AI-assisted strategy research.
- Statistical validation.
- Portfolio construction and risk.
- Paper trading.
- Controlled live execution.
- Monitoring, audit and governance.

3. What We Are and Are Not Building

WE ARE building a research and execution infrastructure company that uses AI to increase research throughput while preserving rigorous quantitative controls.

WE ARE NOT building a chatbot that predicts tomorrow's stock price. We are not assuming LLM outputs are profitable signals, and we are not allowing an LLM to bypass risk controls.

Initially this is proprietary research infrastructure. If later offered to external users, the product, permissions, disclosures, custody model and regulatory requirements must be redesigned.

- Never deploy AI-generated strategies directly to production.

- Never evaluate strategies only in-sample.
- Never ignore fees, spread, slippage, liquidity, latency or market impact.
- Never use data that creates look-ahead or survivorship bias.
- Never let a strategy disable its own risk controls.
- Never expose production trading credentials to research notebooks.

4. Business Model and Strategic Wedge

The realistic first company is an AI Quant Research Lab that can eventually become a proprietary trading operation.

The software-first approach reduces initial capital requirements. The company proves the research process before deploying meaningful capital.

Potential future models include proprietary trading, licensing research infrastructure, enterprise research tooling, or a hybrid structure. Each has different legal and operational requirements.

- Phase 1: internal research infrastructure.
- Phase 2: paper trading.
- Phase 3: proprietary capital with strict limits.
- Phase 4: institutional partnerships and additional markets.
- Phase 5: external technology/capital products only after legal and regulatory review.

5. Target Users and Initial Market

The first internal user is a quant researcher or technically strong founder. The product should optimize for research speed, reproducibility and evidence quality.

Initial market selection should consider data availability, liquidity, execution access, regulatory complexity and team expertise. An India-focused launch could begin with liquid NSE instruments and supported paper trading, subject to current rules.

- Quant researcher.
- Quantitative engineer.
- Risk/trading operator.
- Long-term institutional research/trading team.

6. System Architecture

Separate research, AI, risk and execution. This separation is a core safety principle.

- Market data feed → ingestion → historical/real-time stores → research platform → AI/quant/simulation → backtester → validation → risk → paper trading → human approval → execution gateway → broker/API → exchange.
- Research plane: data → experiments → backtests → reports.
- Control plane: identity → permissions → approvals → audit.
- Trading plane: market data → signal → portfolio → risk → order → broker → exchange.
- AI plane: LLM → tools → research context → experiment runner → results → report.

7. Technology Stack

Use a deliberately observable stack. The moat should come from research quality and execution, not unnecessary framework complexity.

- Python; FastAPI + Pydantic; PostgreSQL; Parquet + object storage; Redis; NumPy/Polars/Pandas/SciPy/scikit-learn; PyTorch/Hugging Face; Qdrant or pgvector; React/Next.js; Docker; OpenTelemetry; Prometheus/Grafana; CI/CD; C++ later for latency-sensitive paths.

8. Repository and Service Structure

A monorepo is reasonable initially because it simplifies shared schemas and local development.

```
ai-quant-lab/
■■■ apps/
■ ■■■ web/ # React/Next.js dashboard
■ ■■■ api/ # FastAPI gateway
■ ■■■ research-worker/ # experiment execution
■■■ services/
■ ■■■ market-data/
■ ■■■ backtester/
■ ■■■ validation/
■ ■■■ portfolio/
■ ■■■ risk/
■ ■■■ paper-trading/
■ ■■■ execution/
■ ■■■ monitoring/
■■■ ai/
■ ■■■ agents/
■ ■■■ prompts/
■ ■■■ tools/
■ ■■■ evaluators/
■ ■■■ model_registry/
■■■ quant/
■ ■■■ strategies/
■ ■■■ features/
■ ■■■ indicators/
■ ■■■ statistics/
■■■ data/
■ ■■■ schemas/
■ ■■■ loaders/
■ ■■■ quality/
■■■ infra/
■ ■■■ docker/
■ ■■■ terraform/
■ ■■■ monitoring/
■■■ tests/
■■■ docs/
■■■ pyproject.toml
```

9. Data Architecture and Data Quality

Data is one of the most important assets. A sophisticated model cannot compensate for incorrect timestamps, missing corporate actions, duplicates, survivorship bias or future information.

Store raw data immutably when licensing permits; build normalized datasets as derived layers; version every dataset.

Use Parquet/object storage for large historical data and PostgreSQL for metadata/transactional state.

- Raw source-preserving layer.
- Normalized layer with standardized symbols/timestamps.
- Feature layer with exact definitions.
- Research snapshots.
- Live real-time state.
- Maintain symbol mappings, corporate actions, splits, dividends, delistings, calendars and time zones.

- Record source, ingestion time, event time, checksum/version, schema version and licensing metadata.

10. Market Data Ingestion

Use event-driven ingestion. Separate source connectors from normalization.

```
MarketDataEvent:
event_id, source, instrument_id, event_time, receive_time,
event_type, price, quantity, bid, ask, sequence_number, payload_version

Source → Connector → Validator → Normalizer → Durable Store
■→ Real-time stream
■→ Data-quality alerts

Checks:
sequence gaps, duplicates, invalid prices/quantities,
stale timestamps, clock drift, out-of-order events,
missing sessions and corporate-action consistency.
```

11. Research and Experimentation Platform

Every experiment must be reproducible. A researcher should be able to reconstruct exactly what happened.

Store strategy version, data version, feature version, parameters, model version, universe, train/test windows, costs, random seed, code commit, environment image and results.

```
Experiment
■■ Hypothesis
■■ Dataset Version
■■ Strategy Version
■■ Feature Version
■■ Parameters
■■ Code Commit
■■ Model Version
■■ Train/Test Windows
■■ Cost Model
■■ Metrics
■■ Artifacts
■■ Approval Status

DRAFT → RUNNING → COMPLETED
■→ REJECTED
■→ NEEDS_REVIEW
■→ PAPER_APPROVED
■→ LIVE_APPROVED
```

12. AI Research Architecture

The AI layer should be tool-using and constrained. The model receives context and invokes deterministic tools. It must not invent numerical backtest results.

Recommended agents: Research, Data, Strategy/Coding, Experiment, Statistical Review, Risk Review and Reporting.

Use structured outputs and validate tool arguments/results before proceeding.

```
User question
↓
Research Agent
■→ Data Tool
■→ Literature/Notes Tool
■→ Feature Catalog Tool
↓
Hypothesis
↓
Strategy/Coding Agent
↓
Sandboxed Experiment Runner
```

```
↓
Backtest Results
↓
Statistical Review Agent
↓
Risk Review Agent
↓
Research Report
↓
Human Approval

AI policy:
- read-only production-data access
- sandboxed code execution
- no broker credentials
- no production DB writes
- deterministic numerical tools
- explicit schemas
- complete audit log
```

13. Strategy Generation

AI-generated strategies are hypotheses, not truths. Strategy specifications must define universe, signal, holding period, sizing, rebalance, risk and execution assumptions.

```
StrategySpec {
name, universe, timeframe, features,
entry_rule, exit_rule, sizing_rule,
rebalance_rule, max_position,
stop_conditions, cost_model, version
}

Example:
Universe: liquid equities
Signal: z-score of spread
Entry: abs(z) > 2
Exit: abs(z) < 0.5
Sizing: volatility scaled
Max position: 2% NAV
Costs: spread + commission + slippage
```

14. Backtesting Engine

The backtester is the scientific engine. It must simulate market events and trading decisions as faithfully as practical.

Start with daily/bar strategies; later support event-driven tick/order-book simulation. Backtests do not guarantee live performance.

- Historical replay.
- Order/fill simulation.
- Cash and position accounting.
- Fees and commissions.
- Bid/ask spread.
- Slippage.
- Latency assumptions.
- Liquidity constraints.
- Partial fills where appropriate.
- Corporate actions.
- Trading calendars.

- Margin rules.
- Portfolio performance.

```
for event in market_events:
update_market_state(event)
strategy.observe(event)
signal = strategy.generate_signal()
target = portfolio.target_position(signal)
orders = portfolio.create_orders(target)
checked = risk.pre_trade_check(orders)
fills = simulator.execute(checked, market_state)
portfolio.apply_fills(fills)
metrics.update(portfolio, event)
```

15. Statistical Validation and Overfitting Controls

A strategy that performs well historically can still be useless. The platform must actively search for reasons to reject it.

Separate development data from evaluation data; never repeatedly tune a locked test set.

- Train/development period.
- Validation period.
- Locked out-of-sample test.
- Walk-forward evaluation.
- Multiple regimes.
- Transaction-cost sensitivity.
- Parameter sensitivity.
- Universe sensitivity.
- Placebo/randomization tests where appropriate.
- Multiple-hypothesis awareness.
- Adjusted performance analysis where appropriate.
- Out-of-sample monitoring after deployment.

```
Approval gate:
1. Data integrity passes.
2. No look-ahead bias.
3. No obvious survivorship bias.
4. Costs included.
5. Out-of-sample performance acceptable.
6. Walk-forward performance acceptable.
7. Risk limits acceptable.
8. Parameter perturbations remain reasonable.
9. Independent review complete.
10. Paper behavior matches simulation within expectations.
```

16. Risk Engine

The risk engine is independent. Strategies cannot modify their own limits.

- Gross/net exposure.
- Per-instrument limits.
- Group/sector exposure.
- Leverage.
- Daily loss.
- Drawdown.

- Order notional.
- Liquidity.
- Concentration.
- Margin.
- Strategy kill switch.
- Global emergency kill switch.

```
Strategy → Proposed Order → Risk Engine
■→ ALLOW → Execution
■→ BLOCK

Emergency:
GLOBAL_KILL = true
Stop new orders
Cancel/neutralize according to runbook
Alert operator
Preserve state
Start incident procedure
```

17. Portfolio Construction

Individual strategies are not the portfolio. Start simple and add optimization only when evidence supports it.

- Equal risk allocation.
- Volatility scaling.
- Correlation-aware allocation.
- Exposure constraints.
- Liquidity-aware sizing.
- Turnover constraints.
- Stress scenarios.
- Drawdown-aware de-risking.

18. Paper Trading

Paper trading bridges research and real execution. Use real-time data and, where practical, the same strategy/risk/execution path as live, replacing broker submission with simulation.

- Real-time data.
- Real-time signals.
- Realistic simulated fills.
- Fees/spread.
- Queue assumptions where possible.
- Monitoring.
- Daily reconciliation.
- Paper-vs-backtest drift analysis.

19. Execution Engine

Execution converts approved target positions into broker orders. It is deterministic, observable and isolated from AI.

- Order validation.

- Idempotency.
- Order state machine.
- Controlled retries.
- Partial fills.
- Cancel/replace.
- Broker acknowledgements.
- Reconciliation.
- Latency measurement.
- Execution-quality analytics.

```
CREATED → RISK_CHECKED → SUBMITTED → ACKNOWLEDGED
→ PARTIALLY_FILLED → FILLED

Alternative states:
REJECTED
CANCEL_REQUESTED → CANCELLED
UNKNOWN → reconciliation required
```

20. Broker and Exchange Connectivity

Initially use supported broker APIs and paper facilities where available. Exact access depends on jurisdiction, account type, market, frequency and current policies.

Never hard-code credentials. Use a secrets manager and separate research credentials from trading credentials.

Institutional scale may require specialized market-data and execution arrangements.

- Market-data entitlement.
- Trading account.
- API credentials.
- Order API.
- Positions/fills API.
- Rate limits.
- Trading calendar.
- Operational escalation.
- Reconciliation.
- Applicable reporting obligations.

21. Live Trading Controls

Live trading must be explicitly approved; strategies move through controlled states.

- Two-person approval where practical.
- Capital allocation limit.
- Data-quality automatic stop.
- Reconciliation automatic stop.
- Risk-breach automatic stop.
- Global kill switch.
- Immutable audit trail.

```
RESEARCH → VALIDATED → PAPER_APPROVED → LIVE_PENDING
→ LIVE_APPROVED → ACTIVE

ACTIVE → PAUSED / KILLED / RETIRED
```

22. Infrastructure and Deployment

Use infrastructure-as-code and separate environments.

- LOCAL → DEV → STAGING → PAPER → PRODUCTION
- Dockerize services.
- Managed PostgreSQL where appropriate.
- Object storage for datasets/artifacts.
- Private networking.
- Least-privilege IAM.
- Backups and recovery tests.
- CI for unit, integration, data-contract and backtest regression tests.

23. Security

Trading credentials are high-value secrets. Security must be designed before live trading.

- Least-privilege IAM.
- Separate AI/research and execution credentials.
- Secrets manager.
- Network segmentation.
- Encryption at rest/in transit.
- MFA.
- RBAC.
- Immutable audit logs.
- Code review for execution changes.
- Sandbox AI-generated code.
- No production secrets for agents.
- Dependency/container scanning.
- Backups/recovery tests.

24. Observability and Incident Response

Every production event must be observable.

- Application/order logs.
- Market-data health.
- Data/order latency.
- Fill/reject rates.
- Risk utilization.
- PnL/drawdown.

- Position reconciliation.
- Service health.
- AI tool calls/model versions.
- Alerts and escalation.

```
Incident:
1. Detect anomaly.
2. Alert operator.
3. Freeze new orders if threshold crossed.
4. Cancel/neutralize according to runbook.
5. Reconcile broker/exchange state.
6. Preserve logs/state.
7. Root-cause analysis.
8. Patch/test.
9. Re-run paper simulation.
10. Approve restart.
```

25. Testing and Quality Assurance

Financial software requires unusually strong testing.

- Unit tests.
- Property-based portfolio tests.
- Backtest regression tests.
- Data-quality tests.
- API integration tests.
- Broker sandbox tests.
- Order-state tests.
- Risk tests.
- Failure injection.
- Load tests.
- Security tests.
- AI evaluation.
- Paper-vs-backtest reconciliation.

26. APIs and Data Contracts

Use versioned schemas and explicit service contracts.

```
POST /experiments
POST /experiments/{id}/run
GET /experiments/{id}
GET /strategies
POST /backtests
GET /backtests/{id}
GET /metrics/{strategy_id}
POST /paper/orders
GET /paper/positions
POST /risk/check
POST /approvals
GET /audit/events

Risk request:
{
"strategy_id": "mr_v7",
"instrument_id": "XYZ",
"side": "BUY",
```

```
"quantity": 100,
"price": 125.50,
"timestamp": "..."
}

Response:
{
"decision": "ALLOW",
"reasons": [],
"limits": {
"max_notional": 500000,
"current_notional": 120000
}
}
```

27. Database Schema

PostgreSQL stores operational/research metadata; large historical datasets belong in object storage/columnar formats.

```
users
roles
permissions
strategies
strategy_versions
datasets
dataset_versions
features
experiments
experiment_runs
backtests
backtest_metrics
model_versions
research_reports
approvals
instruments
orders
fills
positions
risk_limits
risk_events
paper_runs
audit_events
system_incidents

Relationships:
strategy → strategy_versions
strategy_version → experiments
experiment → experiment_runs
experiment_run → backtest_metrics
strategy → approvals
strategy → orders
orders → fills
fills → positions
production actions → audit_events
```

28. Example Research Workflow

Example question: “Does a mean-reversion relationship among a selected liquid universe survive realistic costs and multiple market regimes?”

```
1. Research Agent parses question.
2. Data Agent identifies datasets.
3. Universe Builder creates point-in-time universe.
4. Feature Engine computes spread/z-score.
5. Strategy Agent creates StrategySpec.
6. Experiment Runner executes strategy.
7. Backtester includes costs.
8. Validation runs out-of-sample/walk-forward tests.
```

```
9. Statistical Reviewer checks robustness.
10. Risk Engine computes exposure/drawdown.
11. Report Generator creates report.
12. Human reviewer approves/rejects.
13. Approved strategy enters paper trading.
14. Compare paper results to expectations.
15. Only explicit live approval permits execution.
```

29. Product UX and Dashboard

The dashboard should be built around research decisions, not decorative charts.

- Research inbox.
- Experiment creation.
- Strategy versions.
- Dataset provenance.
- Backtest metrics.
- Equity/drawdown.
- Trade list.
- Cost sensitivity.
- Parameter sensitivity.
- Walk-forward.
- Regime analysis.
- Correlation matrix.
- Risk dashboard.
- Paper/live dashboard.
- Approval workflow.
- Audit log.
- Incident center.

```
Strategy: Mean Reversion v7
Status: PAPER_APPROVED

Performance: CAGR, Sharpe, Max Drawdown,
Volatility, Turnover

Validation: out-of-sample, walk-forward,
cost sensitivity, parameter sensitivity

Risk: gross/net exposure, concentration, liquidity

Operations: last data event, last signal,
last order, reconciliation status
```

30. Team and Hiring Plan

A founding team of 4–6 people is sufficient initially; people can cover multiple roles.

- Founder/Quant Lead — research direction, strategy evaluation, market knowledge, hiring.
- Quant Researcher — hypotheses, statistics, backtesting, reports.
- Quant/Systems Engineer — C++, execution, market data, concurrency, performance.
- AI/ML Engineer — LLMs, agents, evaluation, ML models, research automation.

- Data Engineer — ingestion, normalization, storage, quality.
- Risk/Trading Operator — risk policy, execution, reconciliation, operations.

31. Capital and Budget Planning

Separate software-development cost from trading capital. The MVP can be relatively inexpensive; meaningful trading capital is a separate requirement.

Budget engineering, cloud, data licensing, AI usage, legal/compliance, broker costs, security, monitoring and eventual trading capital.

Start with research and paper trading; keep live capital small until validation and operational testing are complete.

- Data may become a major recurring cost.
- AI costs include model/API usage and experimentation.
- Legal/compliance must be budgeted before live activity.
- Trading capital is not a development budget.
- Maintain an operational runway.

32. Regulatory and Compliance Considerations

Trading rules differ by country, exchange, instrument, account structure and whether the company trades its own money or manages outside money.

For India, obtain professional advice on applicable SEBI, exchange, broker, tax, company-law, data and market-access requirements before live trading or accepting external capital. Exact legal structure must be determined with qualified counsel.

External products can introduce licensing, disclosures, suitability, custody, reporting and cybersecurity obligations.

- Determine proprietary vs regulated client activity.
- Review broker/exchange API terms and market-data licenses.
- Maintain required records.
- Implement access controls and audit trails.
- Have written risk/incident procedures.
- Obtain legal advice before meaningful live deployment.

33. Development Roadmap

Each stage has a gate before the next.

```
Stage 0 — Foundation
Repo, CI/CD, PostgreSQL, object storage, schemas, observability

Stage 1 — Data
Historical loader, normalization, quality, dataset versions

Stage 2 — Backtesting
Strategy interface, simulator, cost model, metrics, experiments

Stage 3 — Research AI
Research agent, tools, hypotheses, sandboxed strategy generation,
experiment runner, reports

Stage 4 — Validation
Walk-forward, out-of-sample, robustness, overfitting controls,
approval workflow
```

```
Stage 5 — Paper Trading
Real-time data, paper execution, reconciliation, monitoring

Stage 6 — Live Readiness
Broker integration, risk engine, kill switches, secrets,
production controls, independent review

Stage 7 — Small Live Deployment
Minimal capital, strict limits, daily reconciliation

Stage 8 — Scale
More strategies/instruments, better execution, C++ latency path,
institutional connectivity
```

34. MVP Definition

The MVP must be small enough to build but serious enough to test the research loop.

- One market/instrument family.
- One or two strategy classes.
- Historical data pipeline.
- Point-in-time universe.
- Backtester.
- Realistic cost model.
- Experiment tracking.
- Research dashboard.
- One LLM research agent.
- Sandboxed strategy generation.
- Out-of-sample validation.
- Paper trading.
- Risk dashboard.
- Audit logs.

```
MVP acceptance:
Ask a research question → generate hypothesis →
run reproducible backtest → inspect costs/risk →
run out-of-sample validation → approve paper trading →
observe paper performance → reproduce the experiment.
```

35. Metrics and KPIs

Track research productivity and trading quality. Do not optimize only for the number of generated strategies.

- Experiments/week.
- Time hypothesis→result.
- Experiment rejection rate.
- Out-of-sample survival rate.
- Paper/backtest drift.
- Data-quality incident rate.
- Reproducibility rate.
- Turnover.

- Cost sensitivity.
- Maximum drawdown.
- Risk-adjusted performance.
- Execution quality.
- Order rejection rate.
- Uptime.
- Incident recovery time.

36. Failure Modes and Lessons

Most failures come from bad assumptions, data, overfitting, operations and uncontrolled complexity.

- Overfitting.
- Look-ahead bias.
- Survivorship bias.
- Data leakage.
- Unrealistic fills.
- Underestimated costs.
- Regime change.
- AI hallucination.
- Tool misuse.
- Stale data/clock/broker failure.
- Risk-limit failure.
- Credential leakage.
- Complexity without evidence.

37. Expansion Strategy

After the first market and strategy family are validated, expand incrementally.

- More strategy families.
- More liquid instruments.
- Futures/options where appropriate.
- Richer microstructure data.
- C++ performance paths.
- Distributed research.
- Model registry.
- Proprietary features.
- Institutional connectivity.
- Potential infrastructure licensing.

38. Long-Term Vision

The long-term goal is an AI-native quantitative organization where research, software, simulation, risk and execution form a continuous feedback loop.

The moat is accumulated proprietary research history, high-quality data, execution knowledge, robust infrastructure and disciplined signal validation.

```
Market Data
↓
AI + Quant Research
↓
Millions of Experiments
↓
Robust Strategies
↓
Risk + Portfolio
↓
Execution
↓
Real-world Outcomes
■■■■■→ Research Feedback → New Hypotheses
```

39. First 90 Days Execution Plan

Designed for a small founding team.

```
Days 1-15:
market scope, data source, repo, PostgreSQL/object storage,
schemas, CI/CD, experiment schema

Days 16-30:
historical ingestion, data quality, dataset versioning,
strategy interface, baseline backtester, metrics

Days 31-45:
transaction costs, portfolio accounting, experiment runner,
research dashboard, strategy versioning

Days 46-60:
LLM research agent, hypothesis schema, sandboxed code generation,
agent→experiment runner, research reports

Days 61-75:
walk-forward, out-of-sample, robustness, risk engine,
approval workflow

Days 76-90:
real-time data, paper trading, reconciliation, alerts,
incident runbooks, end-to-end demo, independent review
```

40. Final Build Checklist

Use this as the readiness checklist.

- Business scope chosen.
- Legal/regulatory review completed for live activity.
- Market-data rights confirmed.
- Broker/exchange access confirmed.
- Historical data validated.
- Point-in-time universe implemented.
- Backtester validated.
- Fees/spread/slippage included.
- Experiment tracking implemented.

- Locked out-of-sample test.
- Walk-forward testing.
- AI code sandboxed.
- AI has no production trading credentials.
- Independent risk engine.
- Global kill switch.
- Paper trading operational.
- Broker reconciliation.
- Production monitoring.
- Secrets management.
- Audit logging.
- Backups tested.
- Incident response documented.
- Human live approval.
- Limited initial capital.
- Independent strategy review.

41. Important Risk Disclaimer

This document is a technical/product-development blueprint, not investment advice, a promise of profitability, or legal/regulatory advice.

Quantitative trading involves substantial risk. Backtested performance does not guarantee future results. AI-generated strategies can be wrong, unstable, overfit or operationally dangerous. Real trading can produce losses beyond expectations, especially with leverage, derivatives, illiquid instruments or faulty execution.

Before live deployment, obtain qualified legal/compliance advice for the target jurisdiction and market, confirm broker/exchange requirements and establish independent risk controls. The safest sequence is research → validation → paper trading → controlled live deployment.

Appendix A — Recommended Initial Technical Stack

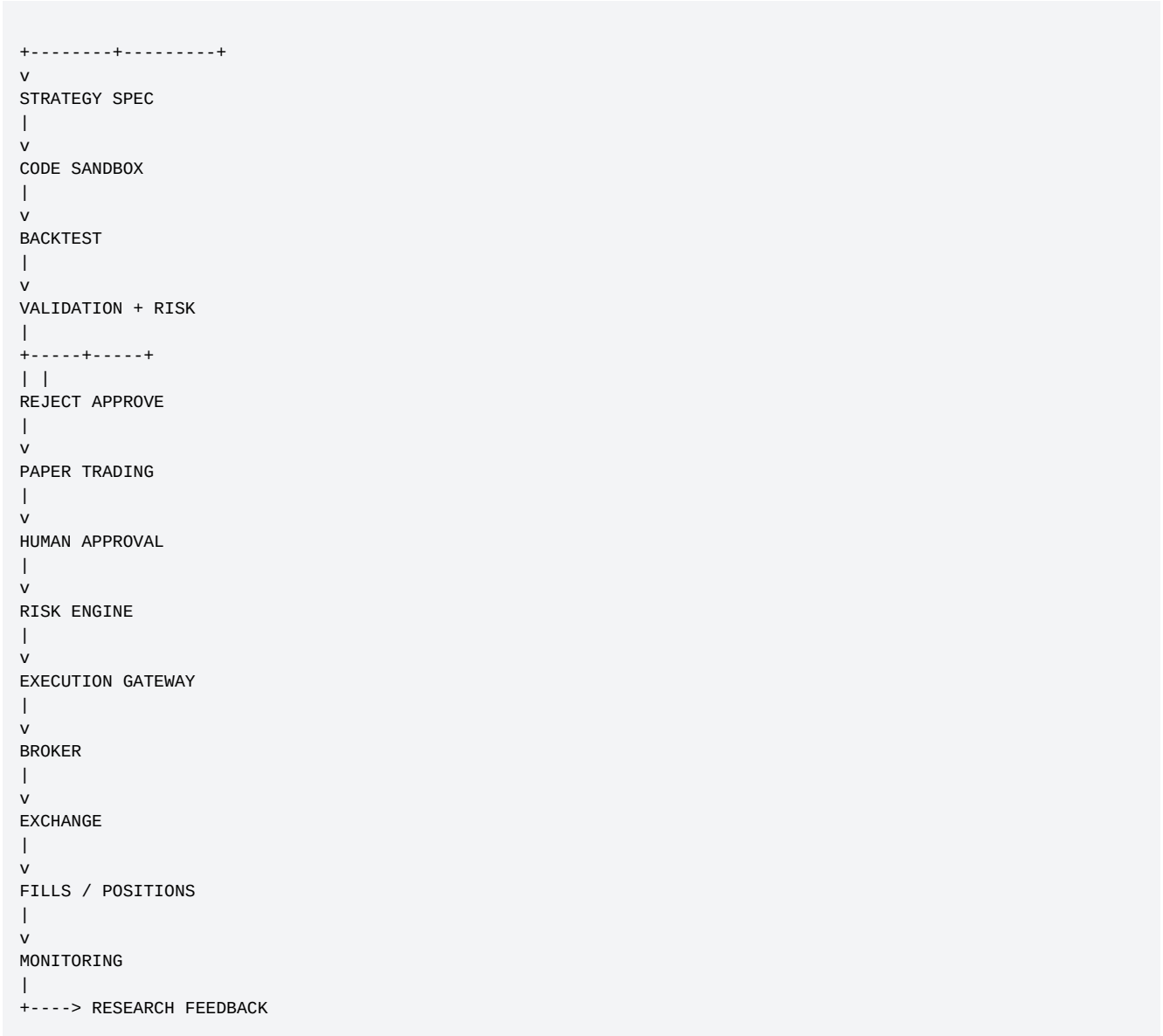
- Python for AI/research.
- FastAPI + Pydantic.
- PostgreSQL.
- Parquet + object storage.
- Redis.
- NumPy/Polars/Pandas/SciPy/scikit-learn.
- PyTorch + Hugging Face.
- Qdrant or pgvector.
- React/Next.js.
- Docker.
- OpenTelemetry + Prometheus/Grafana.
- CI/CD.
- C++ for future low-latency paths.

Appendix B — Core Design Principles

- Evidence over intuition.
- Reproducibility over speed.
- AI assists research; deterministic systems validate it.
- Risk controls are independent.
- Production credentials never enter AI research.
- Historical performance is evidence, not proof.
- Paper trading before meaningful live capital.
- Every production action is auditable.
- Complexity must earn its place.
- Start narrow, prove the edge, then scale.

Appendix C — End-to-End Data Flow

```
SOURCE MARKET DATA
|
v
CONNECTOR
|
v
RAW IMMUTABLE DATA
|
v
NORMALIZATION + QUALITY
|
v
POINT-IN-TIME DATASET
|
+------------------+
| |
v v
FEATURE ENGINE AI RESEARCH
| |
```


+--------+---------+
v
STRATEGY SPEC
|
v
CODE SANDBOX
|
v
BACKTEST
|
v
VALIDATION + RISK
|
+-----+-----+
| |
REJECT APPROVE
|
v
PAPER TRADING
|
v
HUMAN APPROVAL
|
v
RISK ENGINE
|
v
EXECUTION GATEWAY
|
v
BROKER
|
v
EXCHANGE
|
v
FILLS / POSITIONS
|
v
MONITORING
|
+----> RESEARCH FEEDBACK